

[< Go to the original](#)



Talking to Your Data with MCP: No code for data analysis

**Han HELOIR, Ph.D.** ☕

Follow



Data Science Collective a11y-light ~9 min read · May 22, 2025 (Updated: May 22, 2025) · Free: No

Section 1: Burn the Past, Welcome the Now

If you read my previous article, "[Chat Your Data Like You Chat with a Friend](#)" If you liked it — thanks, but sorry. Things move fast in AI — sometimes, painfully fast.

AI doesn't wait. Neither should we. I spent hours crafting that demo, and now it feels like ancient history. My code? Frustrating. My approach? Outdated. But that's what working in AI feels like. One day, you're proud of what you built; the next, you're tearing it down with something ten times better.

The tight coupling of my client-side logic to MongoDB quickly became an anchor, dragging agility down and forcing unnecessary complexity on developer of the solution.

MCP is more than another technology layer — it's the future of data interaction. It's a universal adapter that allows any AI-powered client to talk to your data and use tools effortlessly.

Freedium

ko-fi librepay patreon





Photo by [Shane Rounce](#) on [Unsplash](#)

And yes — I've put it to the test with Anthropic Claude Desktop, which supports MCP clients. This isn't just hype; it's a genuine, game-changing evolution.

This article isn't just an update. It's a reinvention.

Let's dive in.

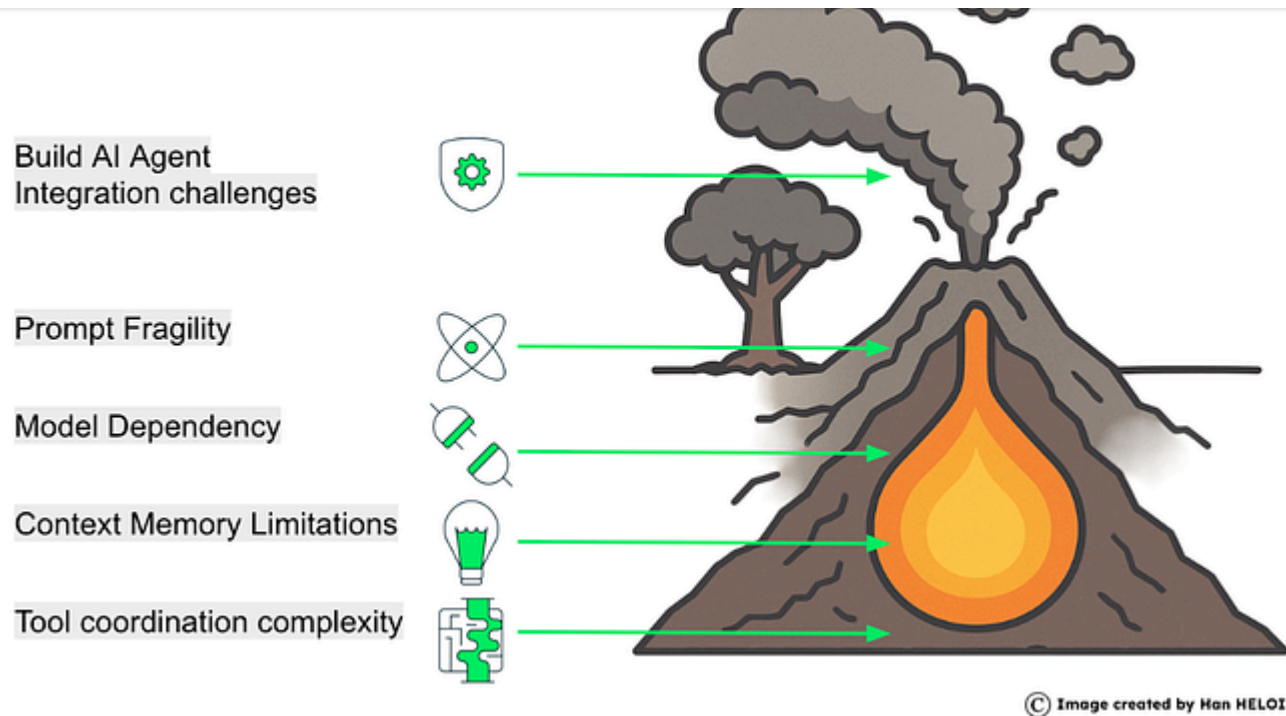
Section 2: AI agent, Tools and why MCP matters

decision-makers at present.

Although these technologies undeniably deserve the community's and ecosystem's attention, **the integration of legacy systems and enterprise line-of-business applications** often receives inadequate focus, despite being a crucial element of the agentic framework.

If you've ever tried building an AI agent that interacts with your data or external tools, you know the painful truth: **the process is messy and the results are often disappointing**. Despite the hype around AI agents, the reality of implementation falls short.

The Reality Check: Why AI Integration Is Harder Than It Looks



Building AI agent Integration Chllenges

1. The House of Cards Called Prompt Engineering

Your carefully constructed prompts are fragile scaffolding that can collapse with the slightest change. When a third-party service updates its API response format or a database schema evolves, your entire integration breaks.

layer to handle these changes gracefully — just brittle prompt instructions that require constant maintenance.

2. Model Roulette: The LLM Dependency Problem

Invested months fine-tuning your MongoDB assistant for GPT-4? Great — now try migrating to Anthropic's Claude or Google's Gemini. You'll essentially start from scratch.

Every model has different capabilities, context handling, and function calling patterns. A database query pattern that works perfectly with one model might produce gibberish with another. This creates effective vendor lock-in, limiting your flexibility to adopt better or more cost-effective models as they emerge.

3. Context Juggling with Limited Working Memory

Every tool connection eats into your model's finite context window. A simple request like "Did anyone comment on my MongoDB performance issue in yesterday's standups?" forces your AI to:

- Identify this as a task requiring Slack history search

- Process potentially lengthy conversation threads

- Extract and summarize the relevant comments

While preserving enough context to provide a coherent response. The model often drops critical details, applies the wrong search criteria, or misinterprets results because it constantly hits context limits.

Side note: If interested in the Agent Memory problem and solutions, you should follow Richmond Alake's work. Here is his LinkedIn.

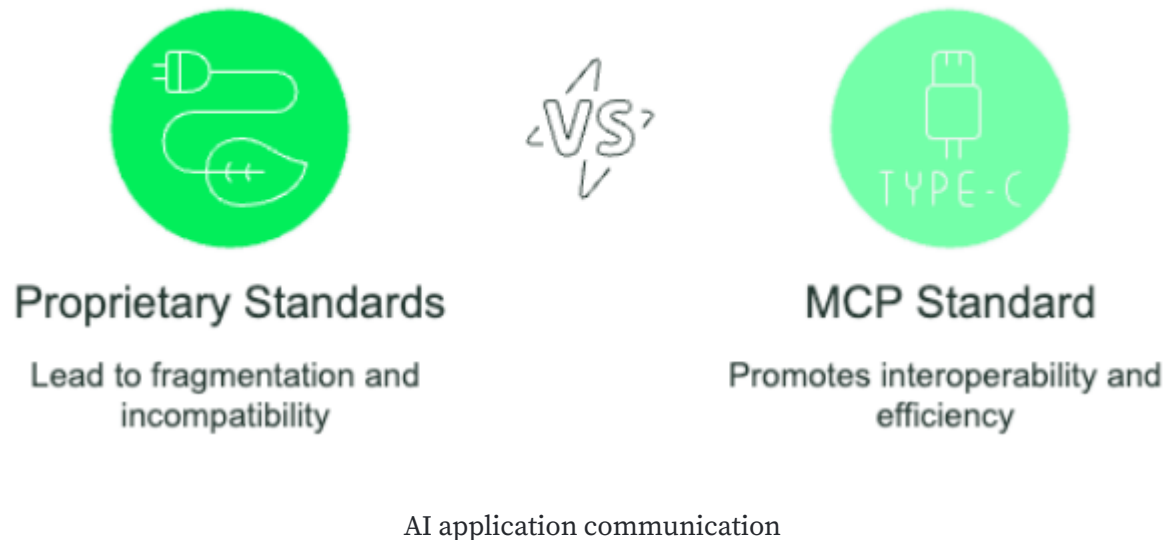
4. The Coordination Challenge

Real-world tasks rarely involve a single action. When your marketing team asks, "How did our last campaign perform compared to Q1 projections?" your AI needs to:

- Query your analytics platform for campaign metrics
- Access financial systems for the original projections
- Perhaps pull customer feedback from your CRM
- Synthesize a coherent analysis from multiple data sources

produced inconsistent outputs.

Here Comes the Model Context Protocol (MCP).



MCP is like the USB-C of AI applications — At its core, MCP is an open standard that creates a standardized communication layer between AI models and the external world.

did for hardware: creating a universal standard that works across the ecosystem.

Section 3: MCP in Practice: Talking to MongoDB

Now that we understand MCP conceptually, let's explore how it transforms the experience of working with MongoDB data.

Traditional approaches required:

Copy

```
// Traditional approach (simplified)
if (user_query.includes("MongoDB") && user_query.includes("sales data")) {
  const template = `You are a MongoDB expert. The database has collections:
                    customers, orders, products. Use function call syntax:
                    query_mongodb({collection, filter})...`;

  // Send template + user query to LLM
  // Parse LLM response, extract function calls
  // Execute MongoDB queries
  // Format results for user
}
```

- Custom code to translate between LLM outputs and MongoDB queries

- **Parsing LLM Responses:** Interpreting the AI's output to extract actionable commands.
- **Executing Database Operations:** Running the derived commands against the databases
- **Formatting Results:** Presenting the data back to the user in a readable format.

With MCP, this complexity disappears. Here's the basic setup:

1. **Register MongoDB as an MCP Tool:** The MCP server exposes your MongoDB instance as a tool, advertising its collections, capabilities, and access patterns.
2. **Connect Claude Desktop:** Since Claude Desktop supports MCP natively, you can simply point it to your MCP server's endpoint.
3. **Start Asking Questions:** With the connection established, you can immediately begin querying your data in natural language — no coding required.

When you ask a question like "What were our top-selling products last quarter?", this happens behind the scenes:

1. Claude interprets your question and recognizes it needs MongoDB data

3. MongoDB executes these operations and returns the results

4. MCP formats the results for Claude

5. Claude presents the answer in natural language

All this happens seamlessly, with no custom code required on your part.

Section 4: From Theory to Practice — talk with your data in MongoDB

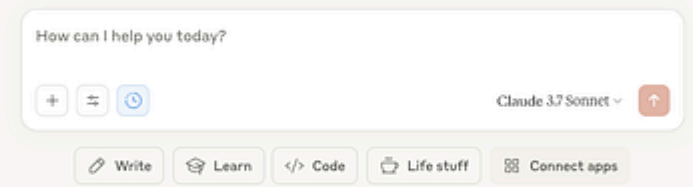
Let's get real.

MCP isn't just a spec. It's a working connector between Claude and my live MongoDB cluster — and the best part?

I wrote zero glue code to make it work.

Step 1: Open Claude Desktop

The experience starts with Claude Desktop — minimal, clean, already personal.

A screenshot of the Claude 3.7 Sonnet chat interface. At the top, it says "Good afternoon, Hanhan" with a small orange star icon. Below that is a text input field with the placeholder "How can I help you today?". To the right of the input field, it says "Claude 3.7 Sonnet" with a dropdown arrow and an orange "Send" button with an upward arrow. Below the input field are five buttons: "Write" (pencil icon), "Learn" (book icon), "Code" (code icon), "Life stuff" (lifebuoy icon), and "Connect apps" (network icon).

This isn't a toy chatbot. Claude can now connect to external tools using **Model Context Protocol**.

Step 2: Plug in MongoDB with MCP

Then open the settings, click on Edit Config, and you will see a file called "claude_desktop_config.json." Open it with any code editor at your preference.



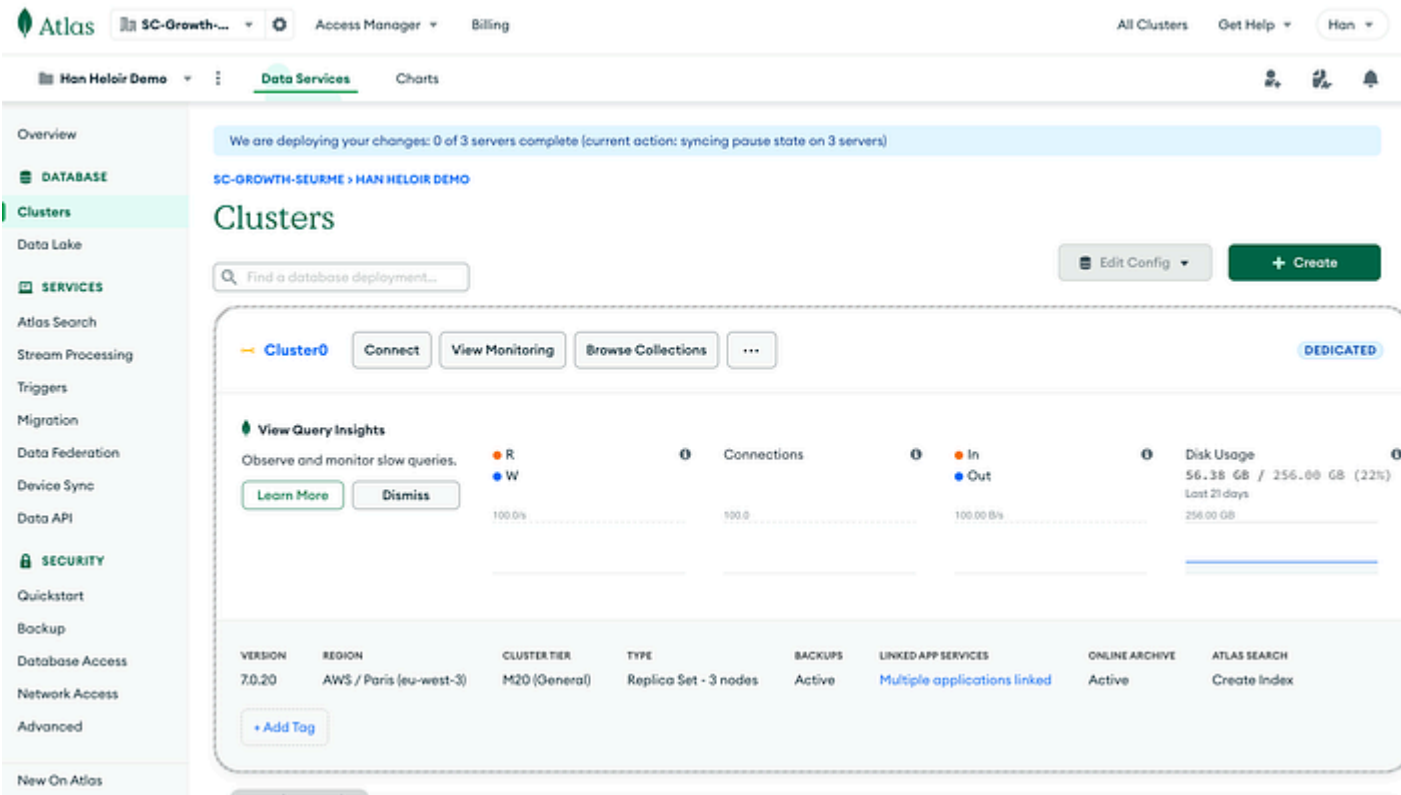
Copy

```
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": [
        "-y",
        "@modelcontextprotocol/server-filesystem",
        "/Users/USERNAME/Desktop",
        "/Users/USERNAME/Downloads"
      ]
    },
    "MongoDB": {
      "command": "npx",
      "args": [
        "-y",
        "mongodb-mcp-server",
        "--connectionString",

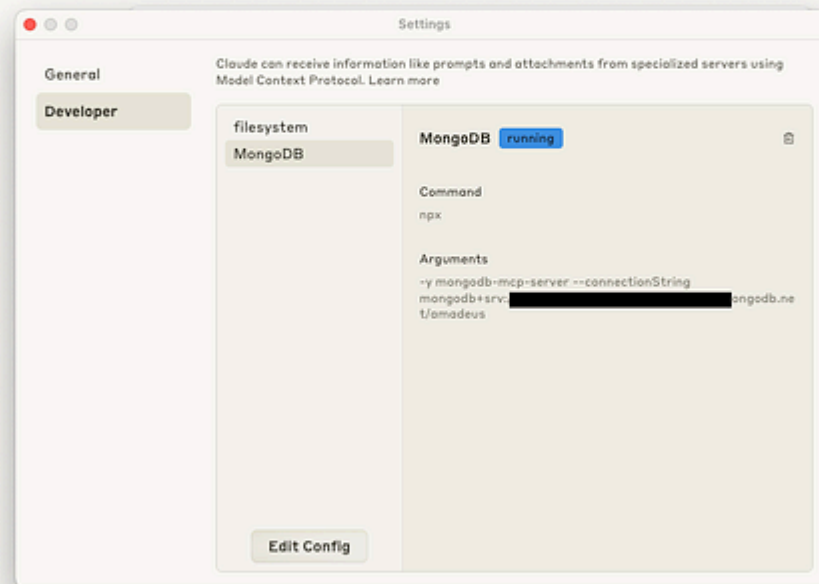
```

```
}  
}
```

Then restart your Anthropic desktop client to make the magic happen.



☀️ Good afternoon, Hanhan



Step 3: Just Ask — Natural Language Queries to Live Data

"List me the databases in MongoDB."

Claude parses it, routes the intent to the correct tool (list-databases), and returns structured results — no custom parser, no chains.

Want to drill in? Just keep chatting:

"Let's look into MCP_analytics. I am launching a new luxe product campaign, I need to know about our client to know how to adresse them on my new product campagne"

That's where Claude really kicked into analyst mode.

Claude, the Analyst: Turning Data into Strategy

Claude first analyzed the customers collection:

- Looked at the **schema**
- Fetched a **sample of documents**
- Isolated **Gold and Platinum** tier clients

"Let's get a count of customers by tier to understand your premium distribution."

"Let's explore the most valued benefits among your luxury clients."

"Let's analyze age distribution."

"Let's understand transaction volume for these segments."

adresse them on my new product campagne

Scrutinizing customer data for luxury product campaign insights

🕒 1s ▾

I'll help you gather customer insights for your luxury product campaign. Let's first examine the schema of the customers collection to understand what data is available.

C collection-schema ▾

Now, let's look at a sample of customer data to understand our client base better:

F find ▾

Let's focus specifically on higher-tier customers who would be most likely to be interested in luxury products. Let's identify Platinum and Gold tier customers:

A aggregate ▾

Let's get a count of customers by tier to understand our premium customer distribution:

A aggregate ▾

Now, let's look at the most common benefits associated with our premium tiers to understand what our luxury customers value:

A aggregate ▾

Let's also check the age distribution of our premium customers to better target the campaign:

A aggregate ▾

Let's try a different approach to get age ranges:

A aggregate ▾

Let's get a summary of the age distribution of our premium customers:

A aggregate ▾

Let's get some transaction data to understand spending patterns:

F find

Let's analyze transaction data for premium customers:

F find

Results: Actionable Insights, Zero Prompt Engineering

Here's what Claude built — by itself — from natural interaction and a live MongoDB backend:

Premium Client Insights for Luxury Product Campaign

Customer Tiers and Distribution

Your customer database has a balanced distribution across tiers:

- **Platinum Tier:** 121 customers (your most valuable audience)
- **Gold Tier:** 112 customers
- **Silver Tier:** 114 customers
- **Bronze Tier:** 109 customers

Premium Customer Demographics

Your luxury tier customers (Platinum and Gold) are primarily:

- **Age Range:** Mostly 45-60 years old with significant wealth
- **Financial Profile:** Active investors with diversified portfolios
- **Investment Behavior:** Engaged in premium financial products and services
- **Preferred Benefits:** Value personalized service and exclusive access

Premium Customer Preferences

The most valued benefits among your luxury customers are:

1. Dedicated account representatives (personalized service)
2. Airline lounge access (travel perks)
3. Concert tickets (entertainment experiences)
4. 24-hour dedicated support lines (immediate assistance)

Account & Investment Characteristics

Premium clients tend to:

- Have multiple accounts (average 3-5 accounts per customer)
- Engage in higher-value transactions (many over \$100,000)
- Invest in diverse assets including tech stocks, commodities, and derivatives
- Utilize premium financial services like Investment Stock, Brokerage, and Commodity products

Recommendations for Your Luxury Campaign

2. **Experience-Based Messaging:** Highlight experiential benefits rather than just material features - these clients value exclusive access and experiences

3. **Sophisticated Communication:** Use refined, knowledgeable language that acknowledges their financial sophistication

4. **Digital & Traditional Channels:** Use both digital communications and traditional high-end print materials - your premium client base spans different age groups with varying channel preferences

5. **Exclusivity Positioning:** Frame your luxury product as an invitation-only offering to appeal to their status-consciousness

6. **Investment Connection:** Consider tying your product to investment or wealth preservation themes if applicable, as this audience is actively engaged in wealth management

7. **Event Integration:** Consider launching your product at exclusive events with dedicated account representatives present to strengthen relationships

Would you like me to elaborate on any specific aspects of these insights for your luxury product campaign?

Premium Client Profile

- **Platinum Tier:** 121 customers
- **Gold Tier:** 112 customers
- **Age:** 45–60
- **Behavior:** Wealthy investors, highly active

They Value

- Personalized account service

- Cultural and entertainment experiences
- Round-the-clock support

Transaction Behavior

- 3–5 accounts per person
- Frequent high-value transfers (\$100K+)
- Investment in tech, commodities, and structured products

Claude's Campaign Recommendations

- Use **refined, intelligent tone**
- Lead with **exclusivity and experience**
- Blend **digital and luxury print** formats
- Consider **private events** with wealth managers

I didn't write a single prompt to do this.

No prompt_template.py, no parse_response(), no chain-of-thought hacks.

1. Claude Desktop
2. An MCP-configured MongoDB tool
3. A natural language chat

From zero to campaign strategy in under 10 minutes.

Final Section: This Is the Future of Talking to Your Data

We've seen what MCP can do: no glue code, no prompt hacks — just real-time answers from live data through a config file and Claude Desktop.

That's not just convenient — it's a game-changer.

Then came **Model Context Protocol (MCP)** — not just a standard, but a **shift in how AI agents connect to tools, data, and business logic.**

With zero custom code, I went from:

"List me the databases in MongoDB"

"Give me campaign-ready insights on my premium customers"

— in a single conversation.

But as Paolo Perrone rightly warns, "**powerful doesn't mean secure.**" We should be aware that MCP introduces some security risks.

If you're building RAG systems, multi-agent workflows, or AI copilots that need **real-time context**, MCP should be on your radar now — not later.

Before you go! 🧑‍🎓

1. If you found value in this article and wish to show your support, **please 'like' this LinkedIn post. You can also find the free friend link in the LinkedIn post.** Your engagement will help extend the reach of this article, your support acts as a great motivator for me. 📝💪❤️
2. Clap my article 50 times, that will really really help me out and boost this article to others. 🙌
3. Follow me on Medium , LinkedIn and subscribe to get my latest article.

Other stories you may be interested in:

Overcoming Security Challenges in Protecting Shared Generative AI Environments

Ensuring Safe AI on Multi-Tenancy

medium.com

Shrinking the Giant: High-Dimensional Vector Efficiency in Production

A Comprehensive Guide to the Vector Quantization

gopubby.com

Maximizing AI Efficiency in Production with Caching: A Cost-Efficient Performance Booster

Unlock the Power of Caching to Scale AI Solutions with LangChain Caching Comprehensive Overview

towardsdatascience.com

The Future of Generative AI is Agentic: What You Need to Know

Implementing AI Agents across LangChain, LlamaIndex, AWS, Gemini, AutoGen, CrewAI and Agent protocol

towardsdatascience.com

The Art of Chunking: Boosting AI Performance in RAG Architectures

The Key to Effective AI-Driven Retrieval

towardsdatascience.com

[#ai](#) [#gen-ai-for-business](#) [#mongodb](#) [#azure](#) [#aws](#)